

EXPERIMENT 1

Containerize and Run a Simple Python Web Application

Objective

To learn how to create a simple Python web application, package it into a Docker image, run it as a Docker container, and access the application from a web browser.

Learning Outcomes

- Create a basic Python Flask web application.
- Create and understand a Dockerfile.
- Build a Docker image from application files.
- Create, start, stop, inspect, and remove a Docker container.
- Map a host port to a container port and access the application through a browser.

Requirements

- Windows 10 or Windows 11 computer.
- Intel Core i5 or equivalent processor.
- 8 GB or 16 GB RAM.
- At least 512 GB storage.
- Docker Desktop installed and running.
- Visual Studio Code installed.
- Internet connection for downloading Docker base images and Python packages.

Important: Commands shown in code boxes are intended for PowerShell unless another location is explicitly mentioned. Application code and Dockerfile content are entered in Visual Studio Code.

PART 1 — INSTALL / CHECK DOCKER

Step 1: Open PowerShell

Open the Windows PowerShell application.

Windows Key

PowerShell

- Open the Windows Start menu using the Windows key.
- Search for PowerShell.
- Open Windows PowerShell.
- A prompt similar to PS C:\Users\Student> should appear.

PRE-LAB SETUP — DOCKER DESKTOP

Complete this setup before starting Experiment 1. If Docker Desktop is already installed and working, students can follow the YES path and continue the lab.

```
Windows PC/Laptop
↓
Is Docker installed?
↙ ↘
YES NO
↓ ↓
docker --version Open Browser
↓ ↓
Continue lab Search Docker Desktop
↓
Download & Install
↓
Open Docker Desktop
↓
PowerShell
↓
docker --version
↓
docker run hello-world
↓
Start the lab
```

If Docker is NOT installed

- Open Google Chrome or Microsoft Edge.
- Search for Docker Desktop for Windows and open the official Docker website.
- Download the Docker Desktop installer for Windows.
- Run the downloaded installer and follow the installation instructions.
- If the installer asks about WSL 2 or related Windows components, allow the required option and restart the computer if requested.
- After installation, open Docker Desktop and wait for Docker to finish starting.
- Open PowerShell and verify the installation using the commands below.

Verify Docker in PowerShell

```
docker --version
```

Then run the Docker test container:

```
docker run hello-world
```

Expected result

```
Hello from Docker!
```

Step 2: Check the Docker installation

Run the Docker version command in PowerShell.

```
docker --version
```

Expected result:

```
Docker version 29.x.x
```

- PowerShell sends the command to Docker.
- Docker returns the installed Docker CLI version.
- This confirms that the Docker command is available.

Step 3: Test Docker with Hello World

Run Docker's test image to verify that Docker can download and start a container.

```
docker run hello-world
```

Expected result:

```
Hello from Docker!
```

- Docker checks whether the hello-world image is available locally.
- If it is not available, Docker downloads it from a registry.
- Docker creates a temporary container from the image.
- The container prints a confirmation message and exits.

PART 2 — CREATE THE PROJECT FOLDER

Step 4: Move to the Desktop

Use PowerShell to move to the Desktop folder.

```
cd Desktop
```

- The cd command means Change Directory.
- The working location changes from the user folder to the Desktop.
- The project will be easy to locate because it is stored on the Desktop.

Step 5: Create the project folder

Create a folder named docker-python-app and enter it.

```
mkdir docker-python-app
```

```
cd docker-python-app
```

Expected result:

```
PS C:\Users\Student\Desktop\docker-python-app>
```

- A new folder is created for the Docker project.
- The second command moves PowerShell into that folder.
- Docker build commands will later use files from this folder.

Important: From this point onward, commands such as `docker build` should be executed from the project folder unless the instructions explicitly say otherwise.

PART 3 — OPEN THE PROJECT IN VISUAL STUDIO CODE

Step 6: Open the project folder in VS Code

Open Visual Studio Code, choose File → Open Folder, and select Desktop → `docker-python-app`. Click Select Folder.

Expected result:

```
EXPLORER

docker-python-app
```

- VS Code will use `docker-python-app` as the project workspace.
- The Explorer panel on the left will show the project files.
- All application and Docker files will be kept together in this folder.

PART 4 — CREATE THE PYTHON APPLICATION

Step 7: Create `app.py`

In the VS Code Explorer, click New File, enter `app.py`, and paste the following code. Save using Ctrl + S.

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello! My first Docker application is running."

app.run(host="0.0.0.0", port=5000)
```

- The file contains the Python web application.
- Flask is used to create a small web server.
- The `/` route returns a message when the browser accesses the application.
- The application listens on port 5000 and on all network interfaces inside the container.

File location: Desktop → `docker-python-app` → `app.py`

PART 5 — CREATE `requirements.txt`

Step 8: Create requirements.txt

In the VS Code Explorer, click New File, enter requirements.txt, paste the following line, and save it.

```
flask
```

- The file lists the Python package required by the application.
- Docker will install Flask while building the image.
- Keeping dependencies in requirements.txt makes the environment reproducible.

Project folder should now contain:

```
docker-python-app
├──
├── app.py
└── requirements.txt
```

PART 6 — CREATE THE DOCKERFILE

Step 9: Create Dockerfile

In VS Code, create a new file named exactly Dockerfile. Do not add .txt to the filename. Paste the following content and save it.

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

- The Dockerfile contains instructions for building the Docker image.
- The base image provides Python 3.12 in a small Linux environment.
- The application files and dependencies are copied into the image.
- The final CMD tells the container how to start the Flask application.

Project folder should now contain:

```
docker-python-app
├──
├── Dockerfile
├── app.py
└── requirements.txt
```

PART 7 — UNDERSTAND WHAT WE CREATED

Application files

```
app.py
↓
Python application

requirements.txt
↓
Required Python libraries

Dockerfile
↓
Instructions for creating the Docker image
```

Meaning of the main Dockerfile instructions

- FROM — selects the starting image, here a lightweight Python environment.
- WORKDIR — creates/selects the working directory inside the image.
- COPY — copies files from the project folder into the image.
- RUN — executes a command while the image is being built; here it installs Flask.
- EXPOSE — documents that the application uses port 5000 inside the container.
- CMD — defines the default command used to start the application.

PART 8 — BUILD THE DOCKER IMAGE

Step 10: Return to the project folder in PowerShell

Open PowerShell and move directly to the project folder.

```
cd "$HOME\Desktop\docker-python-app"
```

```
dir
```

Expected result:

```
Dockerfile
app.py
requirements.txt
```

- The first command ensures PowerShell is in the correct project directory.
- The dir command lists the files in the current folder.
- Docker needs the Dockerfile and application files available in the build context.

Step 11: Build the Docker image

Run the following command from the project folder.

```
docker build -t my-python-app .
```

- Docker reads the Dockerfile.

- Docker downloads the Python base image if it is not already available.
- Docker installs Flask and copies the application into the image.
- The finished image is given the name my-python-app.

Understanding the command

```
docker build
↓
Build an image

-t my-python-app
↓
Give the image the name my-python-app

.
↓
Use the current folder as the build context
```

Step 12: Check the created image

List the Docker images stored on the computer.

```
docker images
```

Expected result:

```
REPOSITORY TAG IMAGE ID SIZE
my-python-app latest xxxxxxxxx xxxMB
```

- Docker displays locally available images.
- The newly created image should appear with repository name my-python-app.
- The tag is normally latest when no other tag is specified.

PART 9 — CREATE AND RUN THE CONTAINER

Step 13: Run the container

In the same PowerShell window, run the image as a background container.

```
docker run -d -p 5000:5000 --name my-python-container my-python-app
```

Expected result:

```
A long value such as:
8f7a1c....
```

- Docker creates a new container from the my-python-app image.
- The container is given the name my-python-container.
- The -d option runs the container in the background.
- The -p 5000:5000 option maps the host's port 5000 to the container's port 5000.

Port mapping

```
Host computer
Port 5000
|
↓
Docker Container
Port 5000
```

Step 14: Check whether the container is running

List currently running Docker containers.

```
docker ps
```

Expected result:

```
CONTAINER ID IMAGE PORTS
xxxxxxx my-python-app 0.0.0.0:5000->5000/tcp
```

- Docker shows containers that are currently running.
- The container named my-python-container should appear.
- The PORTS column should show the 5000-to-5000 port mapping.

PART 10 — OPEN THE APPLICATION

Step 15: Access the application in a browser

Open Google Chrome or Microsoft Edge. Enter the following address in the browser address bar.

```
http://localhost:5000
```

Expected result:

```
Hello! My first Docker application is running.
```

- The browser connects to port 5000 on the student's computer.
- Docker forwards the request to port 5000 inside the container.
- Flask receives the request at the / route.
- The application sends its response back to the browser.

□ The Python application is now running inside a Docker container and is accessible through the browser.

PART 11 — VIEW CONTAINER LOGS

Step 16: View the container logs

Return to PowerShell and display messages produced by the running container.

```
docker ps
```



```
docker logs my-python-container
```

- The first command confirms that the container is running.
- The second command displays the application's output and request logs.
- Logs are useful when checking whether the application started correctly.

PART 12 — STOP THE APPLICATION

Step 17: Stop the container

Stop the running container without deleting it.

```
docker stop my-python-container
```

```
docker ps
```

- The first command stops the running container.
- The container is no longer shown by `docker ps` because it is not running.
- Stopping a container does not automatically remove it.

PART 13 — START THE SAME CONTAINER AGAIN

Step 18: Start the existing container

Start the previously stopped container again.

```
docker start my-python-container
```

```
docker ps
```

- Docker starts the existing container instead of creating a new one.
- The same container name and configuration are retained.
- The application becomes available again on port 5000.

Open the browser again:

```
http://localhost:5000
```

Concept:

STOP

↓

Container still exists

START

↓

Same container runs again

PART 14 — REMOVE THE CONTAINER

Step 19: Stop and remove the container

Stop the container if it is running and then remove it.

```
docker stop my-python-container
```

```
docker rm my-python-container
```

```
docker ps -a
```

- The container is stopped before removal.
- The `docker rm` command permanently removes that container.
- The `-a` option shows both running and stopped containers.
- The image is not removed by `docker rm`.

Step 20: Confirm that the image still exists

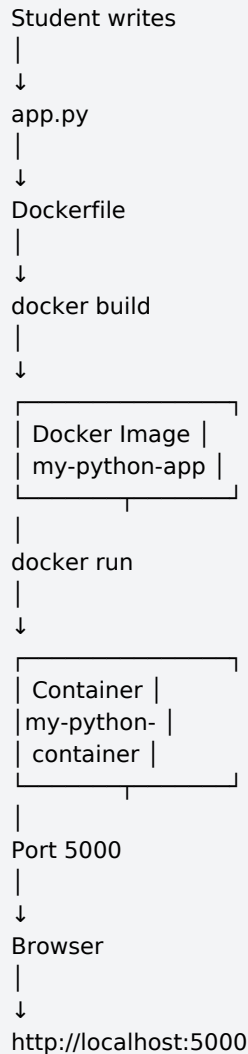
List the Docker images after deleting the container.

```
docker images
```

- The container has been removed.
- The Docker image remains available locally.
- The same image can be used to create another container later.

FINAL ARCHITECTURE

The complete experiment can be understood using the following flow:



PROJECT DESCRIPTION — STEP-BY-STEP UNDERSTANDING

- 1. Project creation: A dedicated folder named docker-python-app is created on the Desktop. Keeping all files in one folder makes the project easy to manage and gives Docker a clear build context.
- 2. Python application: The file app.py contains a small Flask web application. Flask creates the web server, and the root URL / returns a simple message. The application listens on port 5000.
- 3. Dependencies: The requirements.txt file records Flask as the required Python package. This allows Docker to install the dependency automatically while building the image instead of requiring students to install Flask directly on Windows.
- 4. Dockerfile: The Dockerfile describes how the application environment should be created. It starts from a Python base image, selects an internal working directory, copies the dependency file, installs Flask, copies the application, documents port 5000, and defines the command that starts the application.

- 5. Docker image: When `docker build -t my-python-app .` is executed, Docker reads the Dockerfile and creates an image. An image is a packaged, reusable template containing the application and the environment needed to run it.
- 6. Docker container: The command `docker run` creates a container from the image. A container is a running instance of an image. In this experiment the container is named `my-python-container`.
- 7. Port mapping: Flask listens on port 5000 inside the container. The command `-p 5000:5000` connects host port 5000 to container port 5000. This allows a browser running on the student's computer to reach the application.
- 8. Browser access: When the student opens `http://localhost:5000`, the request reaches the host computer's port 5000. Docker forwards the request to the container, Flask processes it, and the response is returned to the browser.
- 9. Container inspection: The `docker ps` command is used to check running containers. It helps students identify the container ID, image name, status, and port mapping.
- 10. Logs: The `docker logs my-python-container` command displays output from the application. Logs are one of the first places to check when an application does not appear to work correctly.
- 11. Stop and start: `docker stop` stops the container but does not delete it. `docker start` starts the same stopped container again. This demonstrates the difference between stopping a container and removing a container.
- 12. Container removal: `docker rm` removes the container. Removing the container does not remove the Docker image. The image remains available for creating another container.
- 13. Overall concept: The complete workflow is source code → Dockerfile → Docker image → Docker container → port mapping → browser. This is the basic foundation of application containerization.
- 14. Why containerization is useful: The application, its dependency, and its runtime environment are packaged together. The same image can be transferred to another computer with Docker and used to create a container there, reducing differences between development and deployment environments.
- 15. What students have achieved: By completing this experiment, students have created an application, described its environment using a Dockerfile, built an image, launched a container, connected a host port to a container port, tested the application, inspected logs, stopped and restarted the container, and finally removed the container.

KEY COMMANDS USED

```
docker --version
docker run hello-world
cd Desktop
mkdir docker-python-app
cd docker-python-app
dir
docker build -t my-python-app .
docker images
docker run -d -p 5000:5000 --name my-python-container my-python-app
docker ps
docker logs my-python-container
docker stop my-python-container
docker start my-python-container
docker rm my-python-container
docker ps -a
```

Expected final result: The student successfully builds a Docker image named my-python-app, runs it as my-python-container, and accesses the Flask application through <http://localhost:5000>.